

PRACTICAL - 2

Programmatic Data Generation, Profiling, and Data Quality Validation for Multiple Data Sources

AIM

To programmatically generate data from different sources such as CSV files, APIs, JSON files, and relational databases, perform schema discovery, profile the incoming data, detect data quality issues, and establish appropriate ingestion criteria before loading data into downstream systems.

PROBLEM DEFINITION

The objective of this practical is to programmatically generate and collect data from multiple sources, including **CSV files, APIs, JSON files, and relational databases**, and prepare it for ingestion into a data engineering pipeline. The system automatically discovers data schemas, profiles the incoming data, and performs data quality validation by identifying issues such as missing values, duplicate records, invalid data types, and schema inconsistencies. Based on these validations, appropriate ingestion criteria are established to ensure that only clean, accurate, and reliable data is processed by downstream systems. This practical helps in understanding how enterprise data engineering pipelines handle diverse data sources while maintaining high data quality and operational efficiency.

Objective

- To generate synthetic datasets from multiple data sources.
- To understand structured, semi-structured, and unstructured data.
- To automatically discover data schemas.
- To perform data profiling and quality validation.
- To detect missing values, duplicate records, and invalid data.
- To prepare clean and validated datasets for ingestion.

Workflow

1. Generate synthetic customer data using the Faker library.
2. Collect data from different sources.
3. Automatically identify schema and data types.
4. Profile the dataset.
5. Detect missing values.
6. Detect duplicate records.
7. Validate data formats.
8. Apply data quality rules.
9. Store clean data for downstream processing.

Data Sources Used

Structured Data

- CSV Files
- SQLite/PostgreSQL Database

Semi-Structured Data

- JSON API Responses

Unstructured Data

- Configuration Text Files
- Log Files

Key Questions

1. How can schema drift be managed while handling third-party APIs?

Schema drift can be managed using dynamic schema detection, schema versioning, metadata catalogs, validation rules, and flexible storage formats such as JSON or Parquet. Data pipelines should automatically detect schema changes and handle new fields without breaking the ingestion process.

2. What are the advantages and trade-offs of using semi-structured JSON over normalized SQL tables?

JSON provides flexibility for storing nested and changing data structures without frequent schema modifications. It is suitable for APIs and event data. However, SQL databases provide better consistency, indexing, faster joins, and efficient querying for structured data.

3. How is data quality validated during ingestion without affecting performance?

Data quality can be maintained by performing lightweight validation during ingestion, validating only critical fields, using parallel processing, schema enforcement, rule-based validation, and logging invalid records instead of stopping the entire pipeline.

Code and Output

```
pip install faker
```

```
import pandas as pd
```

```
import json
```

```
import sqlite3
```

```
import random
```

```
import logging
```

```
from faker import Faker
```

```
fake = Faker()
```

```
customers = []
```

```
for i in range(1000):
```

```
    customers.append({
```

```
        "CustomerID": i + 1,
```

```
        "Name": fake.name(),
```

```
        "Email": fake.email(),
```

```
"Age": random.randint(18, 65),
"City": fake.city()
})

df = pd.DataFrame(customers)

df.head()
df.shape
df.to_csv("customers.csv", index=False)

print("Customer dataset saved successfully.")
df.loc[5, "Email"] = None

df.loc[20, "Age"] = "Unknown"

df.loc[35, "CustomerID"] = None

df.to_csv("customers.csv", index=False)

df.head(40)
df = pd.read_csv("customers.csv")

df.head(40)
df.info()
df.dtypes
df.describe(include="all")
df.isnull().sum()
schema = df.dtypes
```

```
print(schema)

expected_schema = {

    "CustomerID": "float64",

    "Name": "object",

    "Email": "object",

    "Age": "int64",

    "City": "object"

}

for column in expected_schema:

    print(column)

    print("Expected :", expected_schema[column])

    print("Actual  :", df[column].dtype)

    print()

    missing_pk = df[df["CustomerID"].isnull()]

    missing_pk

    missing_email = df[df["Email"].isnull()]

    missing_email
```

```
invalid_age = df[df["Age"] == "Unknown"]

invalid_age
bad_records = df[

    df["CustomerID"].isnull()

    |

    df["Email"].isnull()

    |

    (df["Age"] == "Unknown")

]

bad_records
bad_records.to_csv("quarantine.csv", index=False)

print("Quarantine file created.")
good_records = df.drop(bad_records.index)

good_records.head(40)
good_records.shape
logging.basicConfig(

    filename="ingestion.log",
```

```
level=logging.INFO,

format="%(%asctime)s - %(levelname)s - %(message)s"

)

logging.info("Customer dataset loaded.")

logging.warning("Missing Email detected.")

logging.warning("Missing CustomerID detected.")

logging.warning("Invalid Age format detected.")

logging.info("Bad records moved to quarantine.")

transactions = []

for i in range(1500):

    transactions.append({

        "transaction_id": i + 1,

        "customer": {

            "id": random.randint(1, 200),

            "name": fake.name()

        },
```

```
"payment": {  
  
    "amount": random.randint(100, 5000),  
  
    "mode": random.choice(["UPI", "Card", "Cash"])  
  
}  
  
}))  
with open("api_logs.json", "w") as file:  
  
    json.dump(transactions, file, indent=4)  
  
print("JSON file created.")  
with open("api_logs.json") as file:  
  
    data = json.load(file)  
    api_df = pd.json_normalize(data)  
  
api_df.head()  
api_df.shape  
api_df.info()  
config = ""  
  
Server=localhost  
  
Port=5432
```

Timeout=30

Cache=True

Random Text

Unknown Line

"""

with open("config.txt", "w") as file:

file.write(config)

with open("config.txt") as file:

lines = file.readlines()

for line in lines:

if "=" in line:

print("Valid :", line.strip())

else:

print("Invalid :", line.strip())

conn = sqlite3.connect("customer.db")

good_records.to_sql(

```
"Customers",

conn,

if_exists="replace",

index=False

)

pd.read_sql("SELECT * FROM Customers", conn)

summary = {

    "Total Records": len(df),

    "Valid Records": len(good_records),

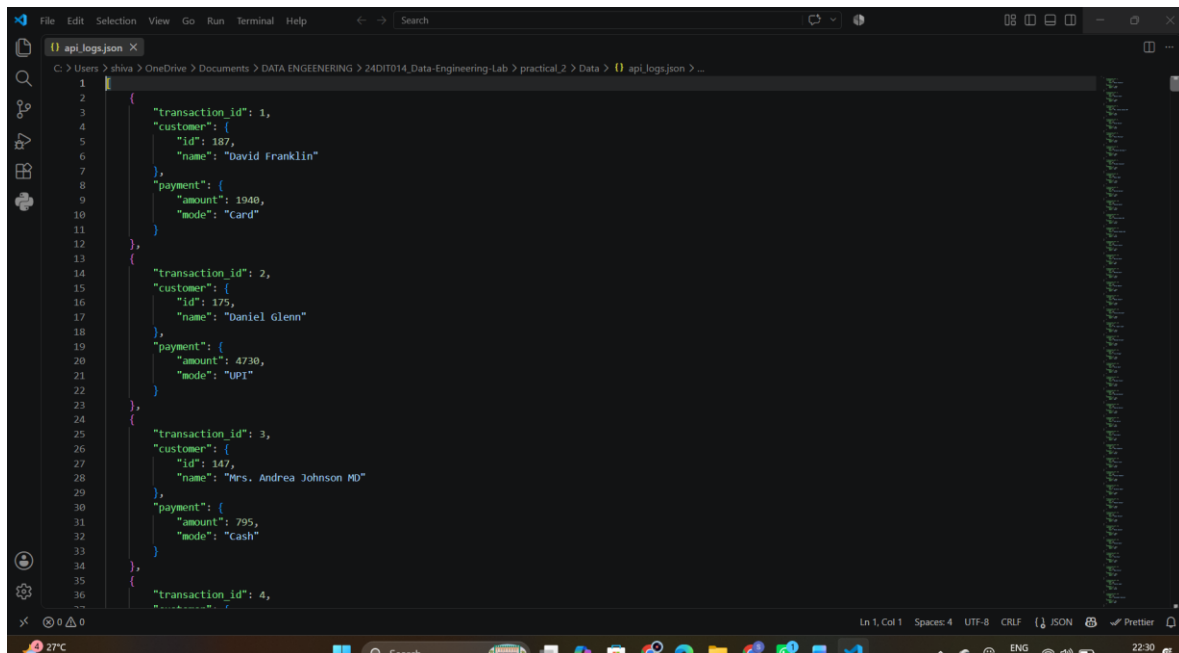
    "Invalid Records": len(bad_records),

    "Missing Email": df["Email"].isnull().sum(),

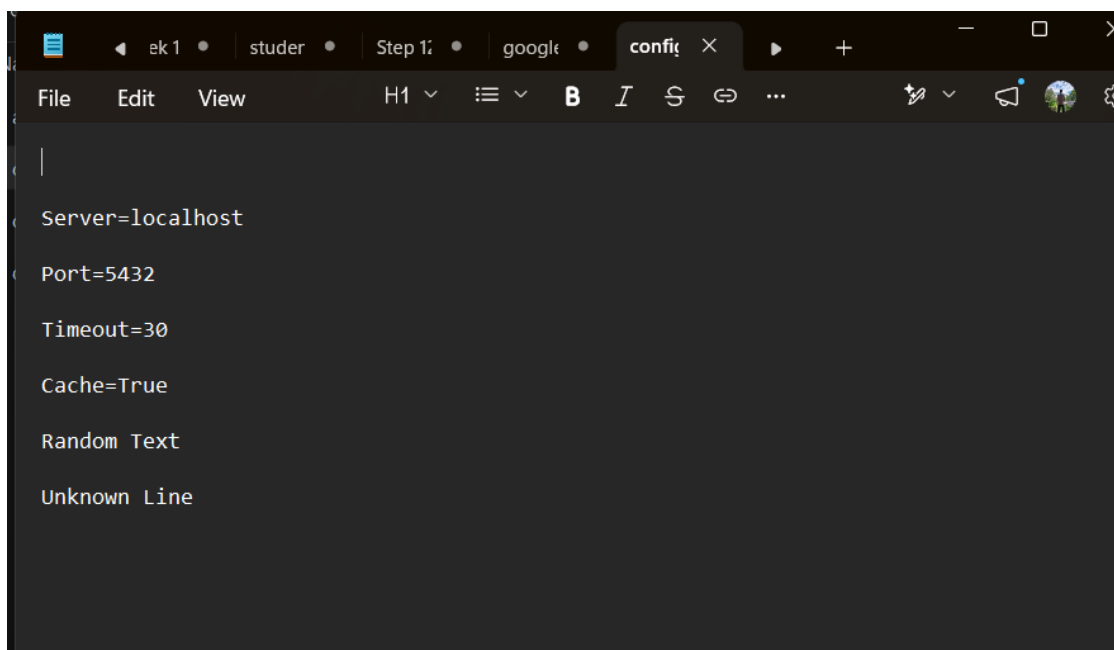
    "Missing CustomerID": df["CustomerID"].isnull().sum()

}
```

```
File Edit View
2026-07-17 09:53:52,566 - INFO - Customer dataset loaded.
2026-07-17 09:53:52,567 - WARNING - Missing Email detected.
2026-07-17 09:53:52,567 - WARNING - Missing CustomerID detected.
2026-07-17 09:53:52,567 - WARNING - Invalid Age format detected.
2026-07-17 09:53:52,567 - INFO - Bad records moved to quarantine.
2026-07-17 09:53:55,697 - INFO - Customer dataset loaded.
2026-07-17 09:53:55,697 - WARNING - Missing Email detected.
2026-07-17 09:53:55,697 - WARNING - Missing CustomerID detected.
2026-07-17 09:53:55,697 - WARNING - Invalid Age format detected.
2026-07-17 09:53:55,698 - INFO - Bad records moved to quarantine.
2026-07-17 10:26:24,296 - ERROR - No such comm target registered: jupyter.widget.control
2026-07-17 10:26:24,298 - WARNING - No such comm: 78b8dfcf-910d-4b55-bec3-24dc641811eb
```



```
File Edit Selection View Go Run Terminal Help
C:\Users\shiva>OneDrive>Documents>DATA ENGINEERING>24DIT014_Data-Engineering-Lab>practical_2>Data>api_logs.json
1
2
3 {
4   "transaction_id": 1,
5   "customer": {
6     "id": 107,
7     "name": "David Franklin"
8   },
9   "payment": {
10    "amount": 1940,
11    "mode": "Card"
12  },
13 },
14 {
15   "transaction_id": 2,
16   "customer": {
17     "id": 175,
18     "name": "Daniel Glenn"
19   },
20   "payment": {
21     "amount": 4730,
22     "mode": "UPI"
23   },
24 },
25 {
26   "transaction_id": 3,
27   "customer": {
28     "id": 142,
29     "name": "Mrs. Andrea Johnson MD"
30   },
31   "payment": {
32     "amount": 795,
33     "mode": "Cash"
34   },
35 },
36 {
37   "transaction_id": 4,
```



```
File Edit View H1 ≡ B I S ↺ ...
Server=localhost
Port=5432
Timeout=30
Cache=True
Random Text
Unknown Line
```

The top screenshot shows an Excel spreadsheet with 26 rows of customer data. The columns are labeled A through W. The data is as follows:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	CustomerID	Name	Email	Age	City																		
2	1.0	Stephanie Reed	smurphy@example.net	52	West Veronicaview																		
3	2.0	Deborah Jackson	leslie13@example.com	21	Port Jamestown																		
4	3.0	David Wagner	mary73@example.com	51	Yorkville																		
5	4.0	Sarah Boyer	hodgegeric@example.net	41	South Melaneshire																		
6	5.0	Julie Hall	huntertimothy@example.org	33	Cuevasmouth																		
7	6.0	Sean Cortez	19	Armstrongchester																			
8	7.0	Sabrina Baker	natalienoores@example.net	41	North Robert																		
9	8.0	Mallory English	claire84@example.net	32	Port Cynthia																		
10	9.0	Timothy Garcia	phillproberts@example.com	46	New Dawn																		
11	10.0	Wanda Reid	aedwards@example.com	60	Port Julie																		
12	11.0	Frank Martinez	grosschristian@example.net	54	West Joshualand																		
13	12.0	Shawn Owens	amiller@example.net	18	Ramirezport																		
14	13.0	Christina Reynolds	danielbaker@example.org	61	Lake Brandonland																		
15	14.0	David Ramos	robymtodd@example.org	44	New Karenberg																		
16	15.0	Mark Carson	balvarez@example.com	26	North Alexandra																		
17	16.0	Rhonda Robertson	sara05@example.net	43	Bradleyland																		
18	17.0	James Smith	ryan09@example.org	47	Jenniferville																		
19	18.0	Kristina Chase	tphillips@example.net	65	Lake Tylerfort																		
20	19.0	Danielle Hall	twilliams@example.org	56	South Lindseyville																		
21	20.0	Shari Gardner	andradesusan@example.net	30	West Derrick																		
22	21.0	Jill Benton	gdavis@example.org	Unknown	Byrdshire																		
23	22.0	Sean Harris	christopher21@example.org	46	Jenniferview																		
24	23.0	William Gibson	jacquelineerowe@example.com	20	East Michaelshire																		
25	24.0	Jonathan Taylor	romeromichael@example.com	55	New Scott																		
26	25.0	Dawn Walker	nicolerodriguez@example.com	39	Stephensville																		

The bottom screenshot shows the same Excel spreadsheet with a filter applied to the 'City' column. Only 4 rows are visible:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	CustomerID	Name	Email	Age	City																		
2	6.0	Sean Cortez	19	Armstrongchester																			
3	21.0	Jill Benton	gdavis@example.org	Unknown	Byrdshire																		
4		Beth Jackson	santanasusan@example.com	57	Hudsonchester																		

GITHUB REPOSITORY

https://github.com/shivanshdalvadidhk-sud/24DIT014_Data-Engineering-Lab/tree/main/practical_2

CONCLUSION

This practical demonstrated how data from multiple sources can be generated, profiled, and validated before ingestion into a data engineering pipeline. Automatic schema discovery and data quality checks helped identify missing values, duplicates, and structural inconsistencies, ensuring that only clean and reliable data moves to downstream systems. The practical provided hands-on experience with enterprise-level data validation techniques and highlighted the importance of data profiling in building robust and efficient data pipelines.